

INHERITANCE

Aim:

To understand and implement inheritance concepts in Java.

PRE LAB EXERCISE

QUESTIONS

✓ What is inheritance?

Inheritance is an Object-Oriented Programming (OOP) concept in which a new class (child/derived class) acquires the properties and methods of an existing class (parent/base class).

It helps in code reuse and improves program structure.

Example:

```
class Result extends Student { }
```

✓ What is code reusability?

Code reusability means using the same code again in different parts of a program without rewriting it.

Inheritance allows the child class to reuse the methods and variables of the parent class, which:

- Reduces code duplication
- Saves time and effort
- Makes programs easier to maintain

✓ What is the use of extends keyword?

The extends keyword is used to inherit a class in Java.

It allows a child class to access the public and protected members of the parent class.

Example:

```
class Result extends Student {  
  
}
```

IN LAB EXERCISE

Objective:

To implement all types of inheritance.

PROGRAMS:

Student Result System (Single Inheritance)

Question:

A school wants to store student details and calculate marks. Create a base class Student and a derived class Result.

Code:

```
class Student {  
    String name;  
    int rollNo;  
  
    void getDetails() {  
        name = "SATHIYA PRIYA M";  
        rollNo = 253;  
    }  
}  
  
class Result extends Student {  
    int marks = 85;  
  
    void display() {  
        System.out.println("Name: " + name);  
        System.out.println("Roll No: " + rollNo);  
        System.out.println("Marks: " + marks);  
    }  
}
```

```

public class Main {

    public static void main(String[] args) {

        Result r = new Result();

        r.getDetails();

        r.display();

    }

}

```

Output:

Name: SATHIYA PRIYA M

Roll No: 253

Marks: 85



```

2  class Student {
3      String name;
4      int rollNo;
5
6  void getDetails() {
7      name = "SATHIYA PRIYA M ";
8      rollNo = 253;
9  }
10 }
11
12 class Result extends Student {
13     int marks = 85;
14
15 void display() {
16     System.out.println("Name: " + name);
17     System.out.println("Roll No: " + rollNo);
18     System.out.println("Marks: " + marks);
19 }
20 }
21 public class Main {
22     public static void main(String[] args) {
23         Result r = new Result();
24         r.getDetails();
25         r.display();
26     }
27 }

```

Problems @ Javadoc Declaration Console × Eclipse IDE for Java Developers 2025-12-R-win32-x86_64

<terminated> Main [Java Application] C:\Users\sanji\Downloads\eclipse-java-2025-12-R-win32-x86_64.exe

Name: SATHIYA PRIYA M
Roll No: 253
Marks: 85

2. Bank Account System (Hierarchical Inheritance)

Question:

A bank has Savings and Current accounts. Both inherit from a common Account class.

Code:

```
class Account {  
    void showAccountType() {  
        System.out.println("Bank Account");  
    }  
}  
  
class SavingsAccount extends Account {  
    void interest() {  
        System.out.println("Savings Account gives interest");  
    }  
}  
  
class CurrentAccount extends Account {  
    void overdraft() {  
        System.out.println("Current Account supports overdraft");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        SavingsAccount s = new SavingsAccount();  
        CurrentAccount c = new CurrentAccount();  
  
        s.showAccountType();  
        s.interest();  
    }  
}
```

```

        c.showAccountType();
        c.overdraft();
    }
}

```

Output:

Bank Account

Savings Account gives interest

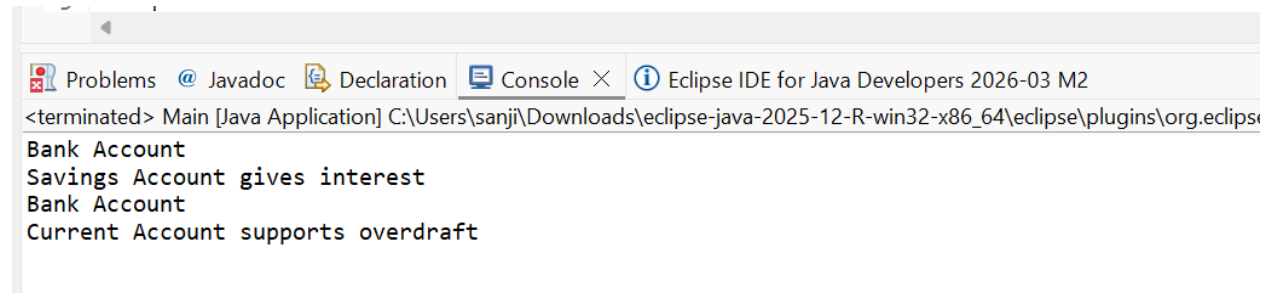
Bank Account

Current Account supports overdraft

```

2  class Account {
3      void showAccountType() {
4          System.out.println("Bank Account");
5      }
6  }
7
8  class SavingsAccount extends Account {
9      void interest() {
10         System.out.println("Savings Account gives interest");
11     }
12 }
13
14 class CurrentAccount extends Account {
15     void overdraft() {
16         System.out.println("Current Account supports overdraft");
17     }
18 }
19
20 public class Main {
21     public static void main(String[] args) {
22         SavingsAccount s = new SavingsAccount();
23         CurrentAccount c = new CurrentAccount();
24
25         s.showAccountType();
26         s.interest();
27
28         c.showAccountType();
29         c.overdraft();
30     }
31 }

```



3. Vehicle System (Multilevel Inheritance)

Question:

A company classifies vehicles as Vehicle → Car → ElectricCar.

Code:

```
class Vehicle {
    void start() {
        System.out.println("Vehicle starts");
    }
}

class Car extends Vehicle {
    void fuelType() {
        System.out.println("Car uses petrol");
    }
}

class ElectricCar extends Car {
    void battery() {
        System.out.println("Electric car uses battery");
    }
}

public class Main {
    public static void main(String[] args) {
        ElectricCar e = new ElectricCar();
        e.start();
        e.fuelType();
        e.battery();
    }
}
```

```
2 class Vehicle {
3     void start() {
4         System.out.println("Vehicle starts");
5     }
6 }
7
8 class Car extends Vehicle {
9     void fuelType() {
10        System.out.println("Car uses petrol");
11    }
12 }
13
14 class ElectricCar extends Car {
15     void battery() {
16        System.out.println("Electric car uses battery");
17    }
18 }
19
20 public class Main {
21     public static void main(String[] args) {
22        ElectricCar e = new ElectricCar();
23        e.start();
24        e.fuelType();
25        e.battery();
26    }
27 }
```

Problems Javadoc Declaration Console Eclipse IDE for Java Developers 2026-03 M2

<terminated> Main [Java Application] C:\Users\sanji\Downloads\eclipse-java-2025-12-R-win32-x86_64\eclipse\plugins\org.eclipse.just

Vehicle starts
Car uses petrol
Electric car uses battery

Output:

Vehicle starts

Car uses petrol

Electric car uses battery

POST LAB EXERCISE

- ✓ Why Java does not support multiple inheritance using classes and how it is implemented?

Java does not support multiple inheritance using classes to avoid ambiguity and complexity, such as the Diamond Problem (when two parent classes have the same method name).

Java supports multiple inheritance using interfaces instead.

Example:

```
interface A {
    void show();
}
```

```
interface B {
    void display();
}
```

```

}

class C implements A, B {
    public void show() {
        System.out.println("Show method");
    }
    public void display() {
        System.out.println("Display method");
    }
}

```

✓ What is the role of the super keyword? Give examples.

The super keyword is used to:

- Access parent class variables
- Call parent class methods
- Invoke parent class constructor

Example:

```

class Student {
    String name = "Anitha";
}

class Result extends Student {
    String name = "Ram";

    void display() {
        System.out.println(super.name); // Access parent class variable
    }
}

```


- ✓ Can a child class access private members of the parent class? Why?

No, a child class cannot directly access private members of the parent class.

Reason:

Private members are encapsulated and accessible only within the same class to protect data.

Access is possible indirectly using public or protected methods (getters/setters).

- ✓ Explain why hybrid inheritance is not supported in Java.

Hybrid inheritance is a combination of two or more types of inheritance (e.g., multiple + multilevel).

Java does not support hybrid inheritance with classes because it can cause:

- Ambiguity
- Diamond problem
- Confusion in method resolution

However, hybrid inheritance can be achieved using interfaces in Java.

Result:

Thus the different types of inheritance were implemented and executed successfully.

ASSESSMENT

Description	Max Marks	Marks Awarded
Pre Lab Exercise	5	
In Lab Exercise	10	
Post Lab Exercise	5	
Viva	10	
Total	30	
Faculty Signature		